Jeudi 05/11 — Flexbox, Grid et mobile first

Kevin Bardoux

jeudi 5 novembre 2026

Table des matières

Objectifs 2
Matin — Flexbox & Grid · mobile first · Flexbox Froggy 3
Rappel (5 min) 3
Apport 1 — Flexbox (10 min) 3
Les valeurs, en images 4
Flexbox — attention et vérification 11
Flexbox Froggy (40 min) 11
Apport 2 — Grid (15 min) 11
Grid — attention et vérification 15
Lire une page de documentation — `justify-content` (20 min) 15
Pas à pas, en captures 15
Quatre questions, en anglais dans le texte 22
Enregistrer et envoyer (2 min) 22
À retenir — ce matin 22
Si le temps le permet — Les derniers niveaux de Froggy . 23
Débrief 23
Après-midi — TP — Grid Garden, puis « Layout Lab » 23
Grid Garden (45 min) 23
Apport — Mobile first (15 min) 24
Mobile first — attention et vérification 26
« Layout Lab » (1 h 30) 27
Enregistrer et envoyer (2 min) 27
Publier sur GitHub Pages (20 min) 27

À retenir — cet après-midi 29
Critères de réussite 29
Si le temps le permet — Publier le défi au portfolio 29
Si le temps le permet — Le formulaire sur grand écran 29
Pour aller plus loin 29
Sources 30

Note

Compétences visées : C3 · communiquer (anglais) **Livrable** : la page personnelle mise en page en *mobile first*, `node --test layout.test.js` au vert.

! Le minimum du jour

- Flexbox Froggy : les 24 niveaux ; Grid Garden : le niveau 12 au moins.
- `node --test layout.test.js` au vert, sans barre de défilement horizontale à 375 px.
- La page personnelle publiée sur GitHub Pages, en HTTPS.

Le reste (rubriques « Si le temps le permet » et encadrés « Défi ») est la suite naturelle, pour ceux qui avancent.

Objectifs

- Aligner et répartir des éléments avec Flexbox.
- Construire une grille avec Grid, et choisir entre les deux.
- Écrire une mise en page *mobile first* avec une media query.
- Publier sa page sur GitHub Pages.

Comment se passe la journée. Le matin, deux notions : Flexbox et Grid ; l'après-midi, *mobile first*, juste avant de mettre en page votre page personnelle, puis la publication sur GitHub Pages. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée.

Matin — Flexbox & Grid · mobile first · Flexbox Froggy

Rappel (5 min)

1. Quelle unité pour une taille de texte accessible ?
2. Comment s'appelle un `push` refusé par GitHub ?
3. Que fait `typeof` ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Flexbox (10 min)

- **Pourquoi** : aligner des éléments sur une ligne, sans calcul.
- **Flexbox** : un parent `display: flex` répartit ses enfants sur un axe.

Pourquoi. Un menu dont les liens s'alignent et se répartissent, une carte dont l'image et le texte se placent côte à côte : avant Flexbox, il fallait des calculs fragiles.

Définition. Flexbox dispose les **enfants** d'un élément sur une dimension, en ligne ou en colonne (MDN Web Docs, s. d.-a). `justify-content` règle l'axe principal, `align-items` l'axe secondaire, `gap` l'espace entre les éléments ; `flex-wrap: wrap` les fait passer à la ligne si la place manque.

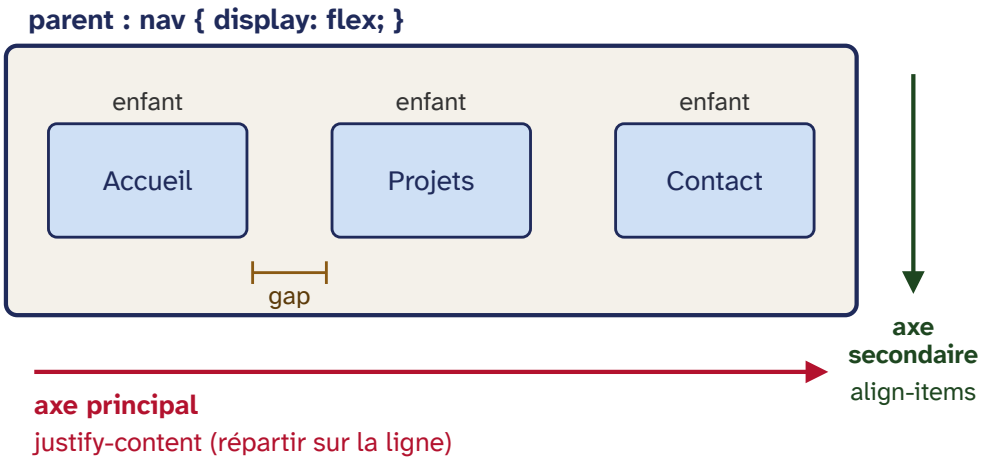


Figure 1. – Flexbox : le parent `display: flex` place ses enfants sur l'axe principal (`justify-content`) ; `align-items` règle l'axe secondaire, `gap` l'espace entre eux.

Démonstration en direct, à ouvrir dans le navigateur : justify-content et align-items sur un menu de trois liens. Elle se règle au clavier (Tab, puis les flèches) ou à la souris.

Exemple.

```
.menu {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```

Pas à pas. Le formateur applique chaque ligne au menu de sa page, l'aperçu ouvert.

1. `display: flex` sur le `nav` : les liens se mettent en ligne.
2. `justify-content: space-between` : ils se répartissent sur toute la largeur.
3. `align-items: center` : ils se centrent en hauteur.
4. `gap: 1rem` : un espace régulier entre eux.

Les valeurs, en images

Chaque vignette est le rendu réel de son code : trois enfants A, B, C dans un parent en pointillés, qui a `display: flex`.

`justify-content` : la répartition sur l'axe principal.



Figure 2. – `flex-start`

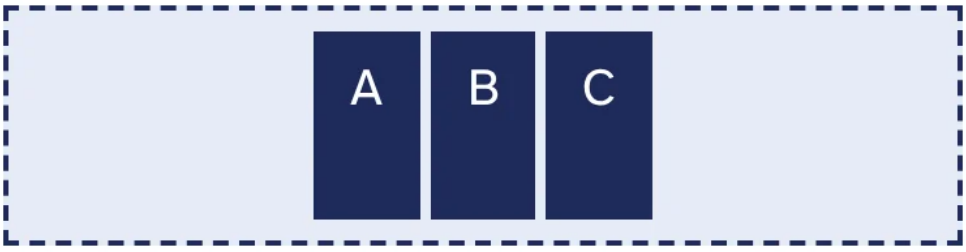


Figure 3. – `center`



Figure 4. – `flex-end`

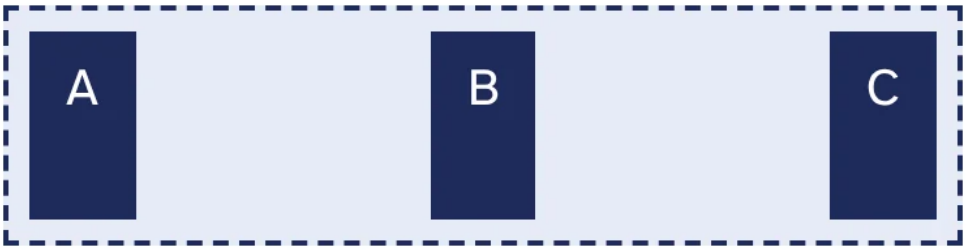


Figure 5. – `space-between`

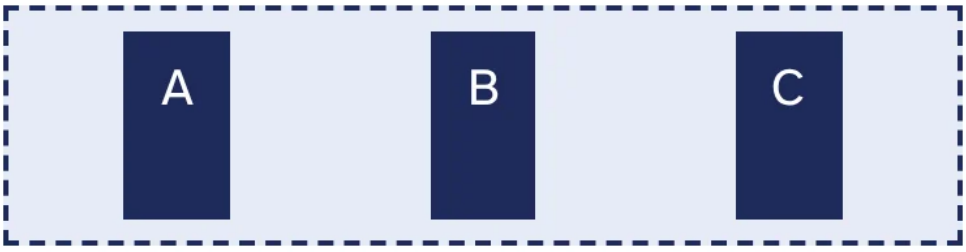


Figure 6. – `space-around`

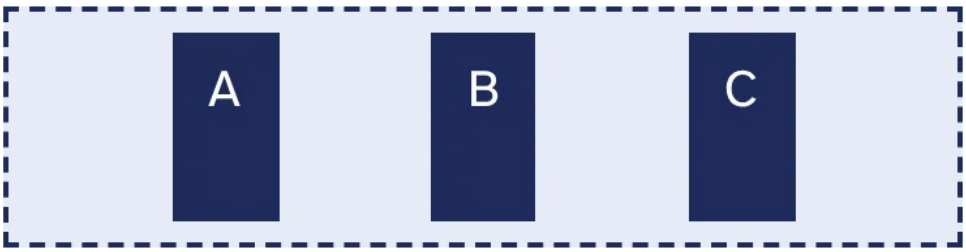


Figure 7. – `space-evenly`

`align-items` : l'alignement sur l'axe secondaire (trois enfants de hauteurs différentes).

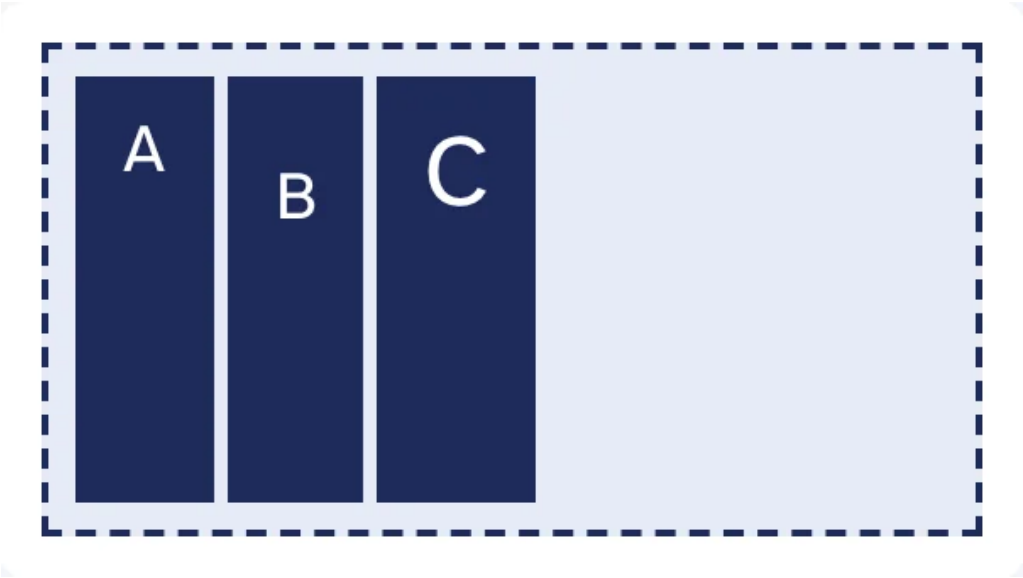


Figure 8. – `stretch`

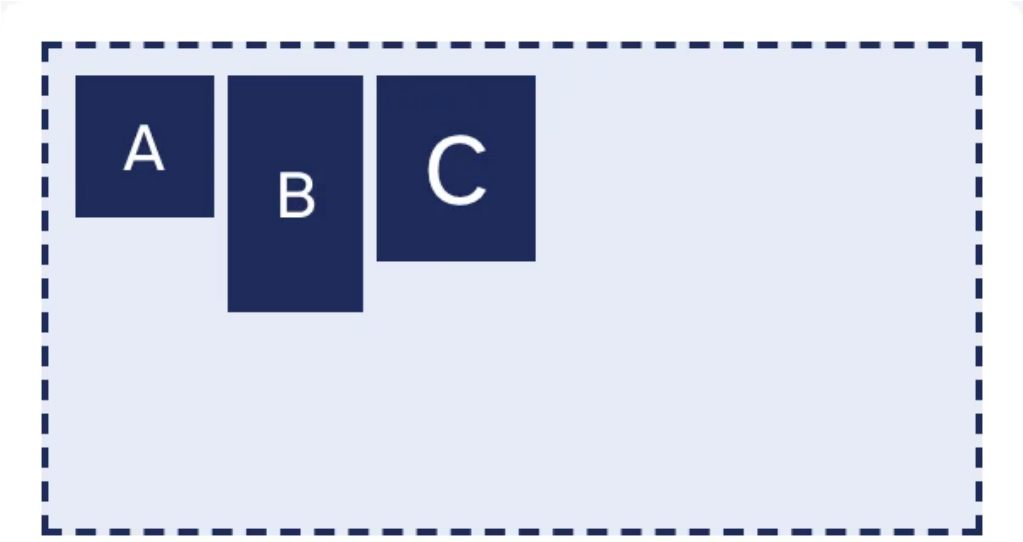


Figure 9. – `flex-start`

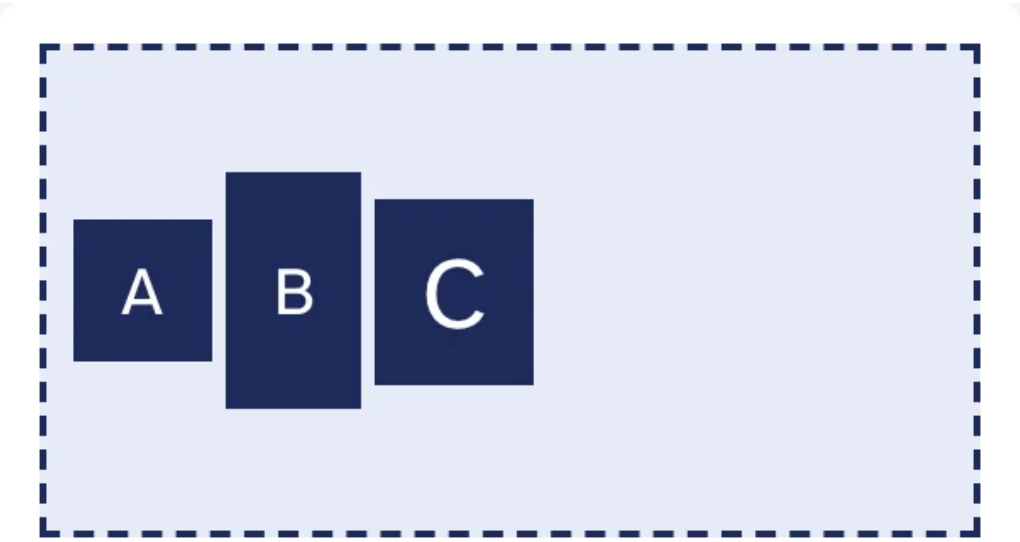


Figure 10. – `center`

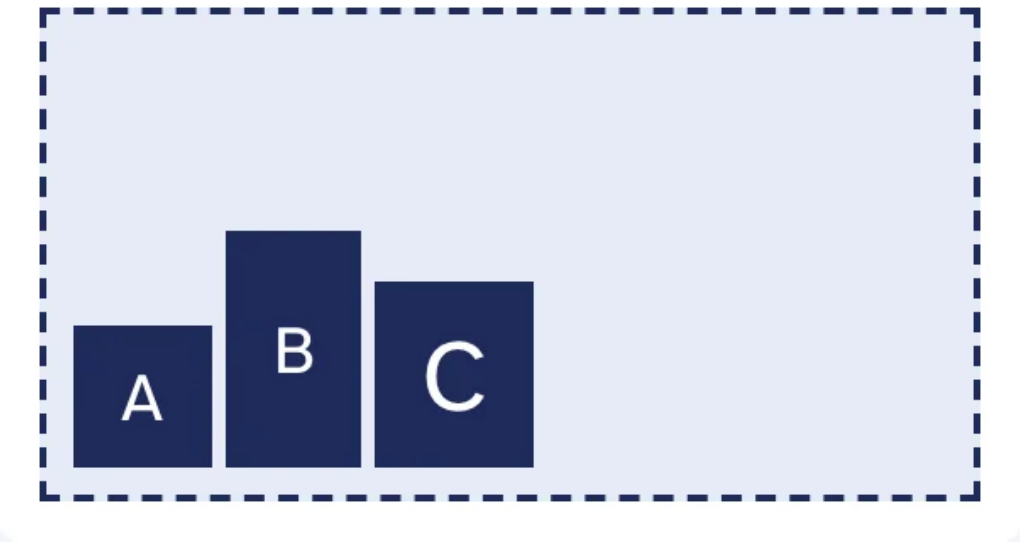


Figure 11. – `flex-end`

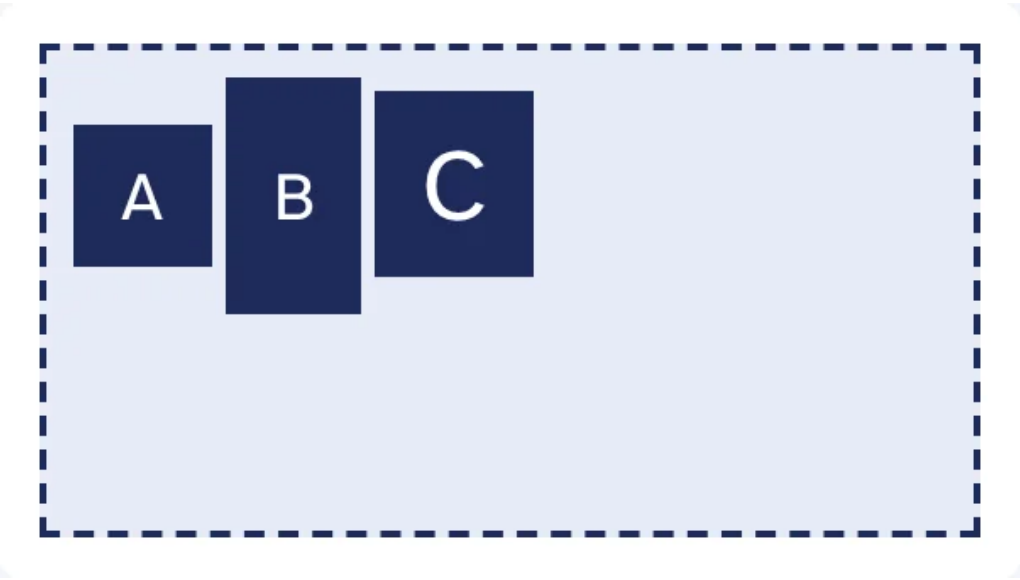


Figure 12. – `baseline`

`flex-direction` : le sens de l'axe principal.



Figure 13. – `row`



Figure 14. – `row-reverse`



Figure 15. – `column`

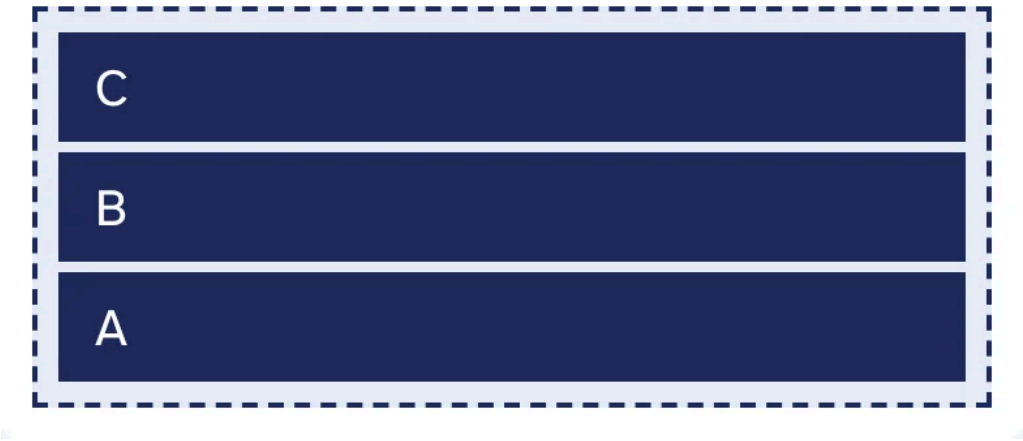


Figure 16. – `column-reverse`

`flex-wrap` : six éléments de 90 px, dans un parent trop étroit.

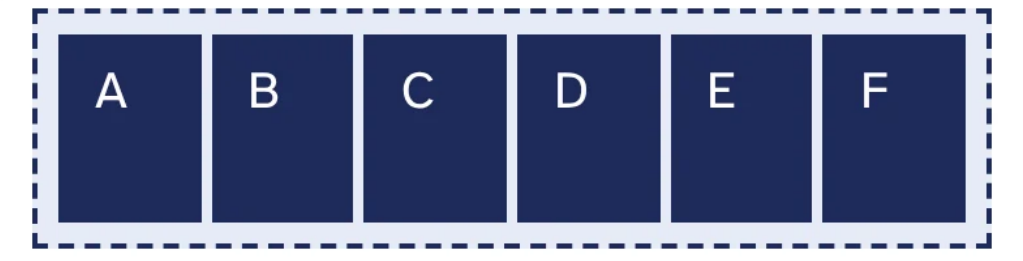


Figure 17. – `nowrap`

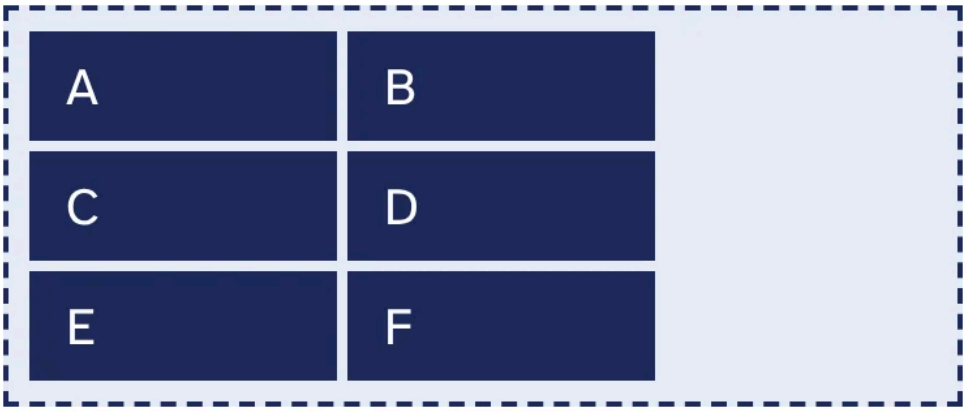


Figure 18. – `wrap`

`gap` : l'espace entre les éléments.

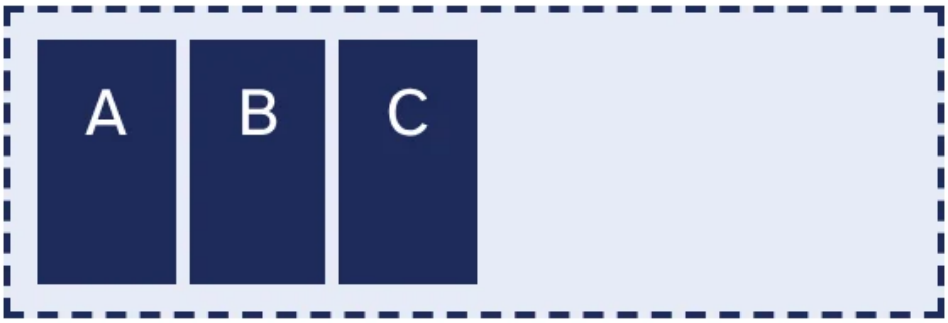


Figure 19. – `0`

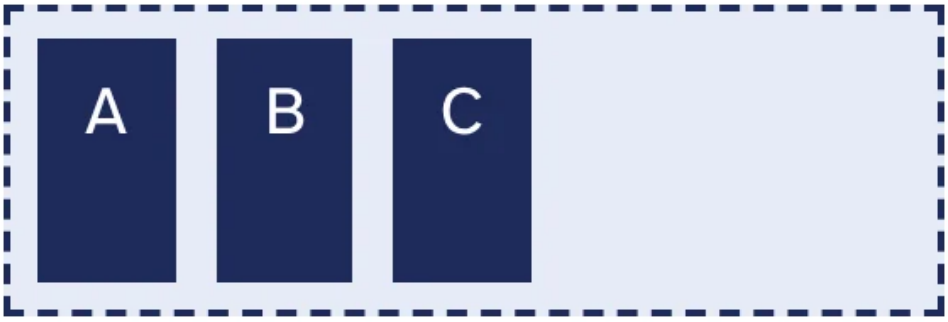


Figure 20. – `0.5rem`

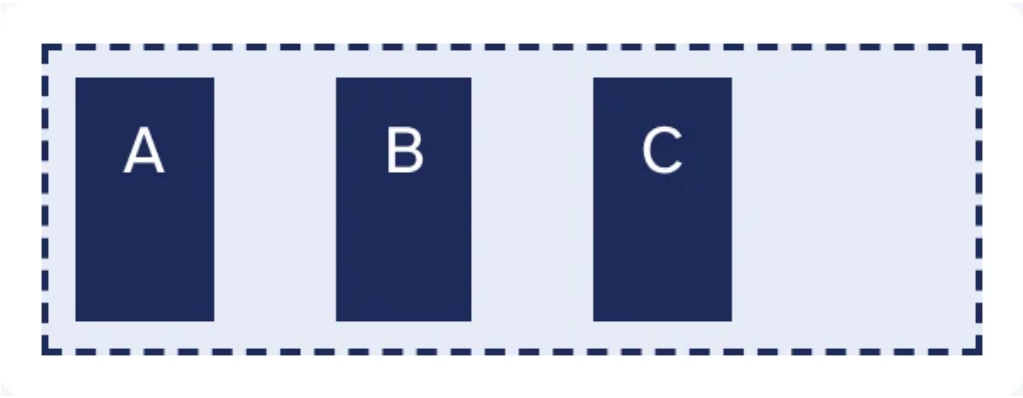


Figure 21. – `2rem`

Flexbox — attention et vérification

> ⚠ Attention
>
> `display: flex` se met sur le **parent** (le `nav`), pas sur les enfants (les liens). Sinon, rien ne bouge.

- **Vérifiez** : quelle propriété pour centrer verticalement ?

Flexbox Froggy (40 min)

1. Ouvrez Flexbox Froggy, version du cours : le jeu s'affiche en français.
2. Placez les grenouilles sur leurs nénuphars, niveau par niveau.
3. Notez chaque propriété découverte.
4. **Réussi si** : les 24 niveaux ; capture du dernier niveau réussi.

Niveau : guidé.

Flexbox Froggy existe en français et compte 24 niveaux (Thomas Park, s. d.-a). Cette copie est hébergée avec le cours, sans mesure d'audience ni publicité ; votre progression reste dans votre navigateur. Code : MIT, © Thomas Park. Images : Creative Commons BY 3.0, © Thomas Park. Propriétés à noter : `justify-content`, `align-items`, `flex-direction`, `order`, `align-self`, `flex-wrap`, `align-content`.

Apport 2 — Grid (15 min)

- **Pourquoi** : ranger des cartes en lignes **et** en colonnes.
- **Grid** : un parent `display: grid` place ses enfants dans une grille.

Pourquoi. Une galerie de projets, une page avec en-tête, contenu et pied : il faut aligner dans les deux sens à la fois.

Définition. Grid organise en **deux** dimensions, lignes et colonnes (MDN Web Docs, s. d.-b, s. d.-d). `grid-template-columns` définit les colonnes ; `1fr` est une fraction de l'espace disponible. **Flexbox ou Grid ?** Flexbox pour une rangée d'éléments (un menu, une liste de badges) ; Grid pour une grille à deux dimensions (des cartes, la page entière). Les deux se combinent : une grille de cartes, un menu en Flexbox.

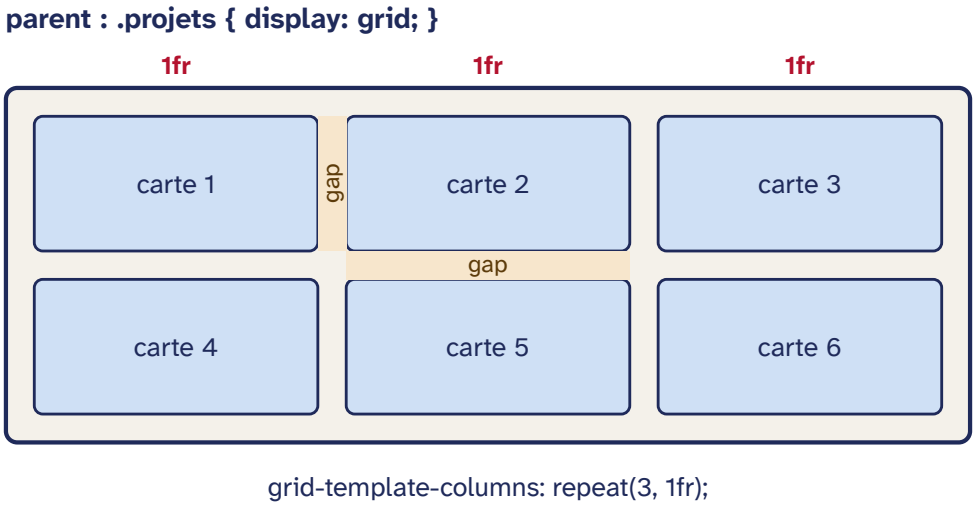


Figure 22. – Grid : `repeat(3, 1fr)` fait trois colonnes égales ; les cartes remplissent la grille ligne par ligne, avec le même `gap` dans les deux sens.

Démonstration en direct, à ouvrir dans le navigateur : grid-template-columns sur six cartes. Elle se règle au clavier (Tab, puis les flèches) ou à la souris.

Exemple.

```
.projets {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

Pas à pas.

1. `display: grid` sur le conteneur des projets.

2. `grid-template-columns: repeat(3, 1fr)` : trois colonnes égales.
3. `gap: 1rem` : l'espace entre les cartes, dans les deux sens.
4. F12 : cliquez sur le badge `grid` à côté de l'élément : la grille se dessine sur la page.

Quelques motifs courants de `grid-template-columns`, sur les mêmes six cartes :

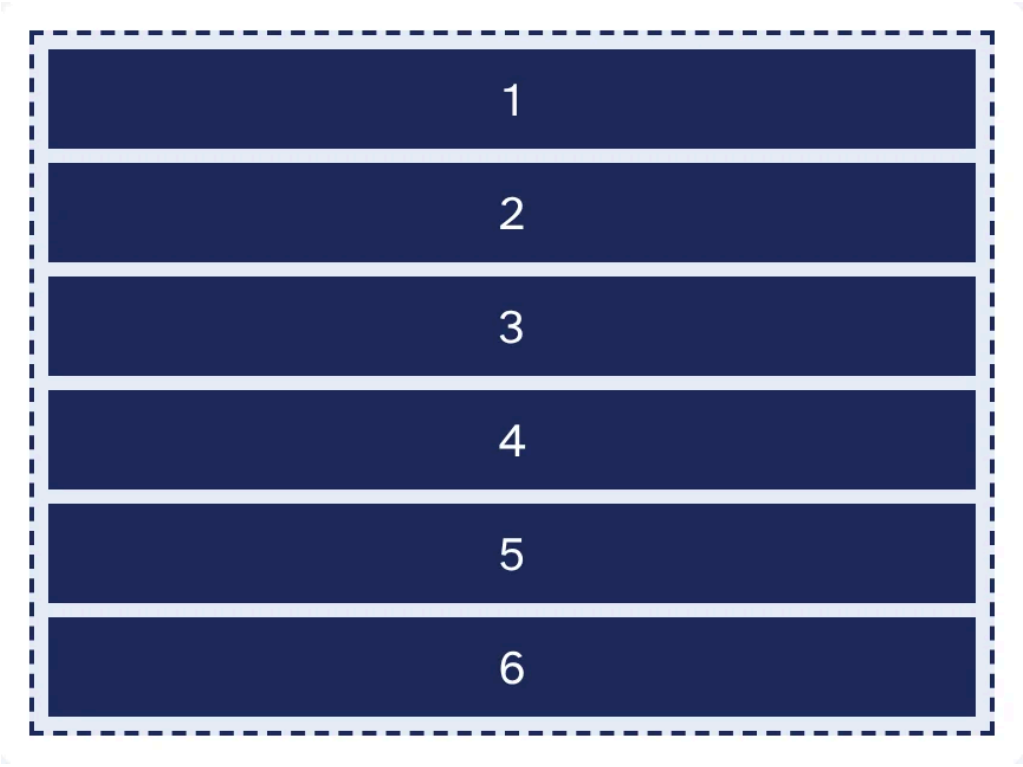


Figure 23. – `1fr`

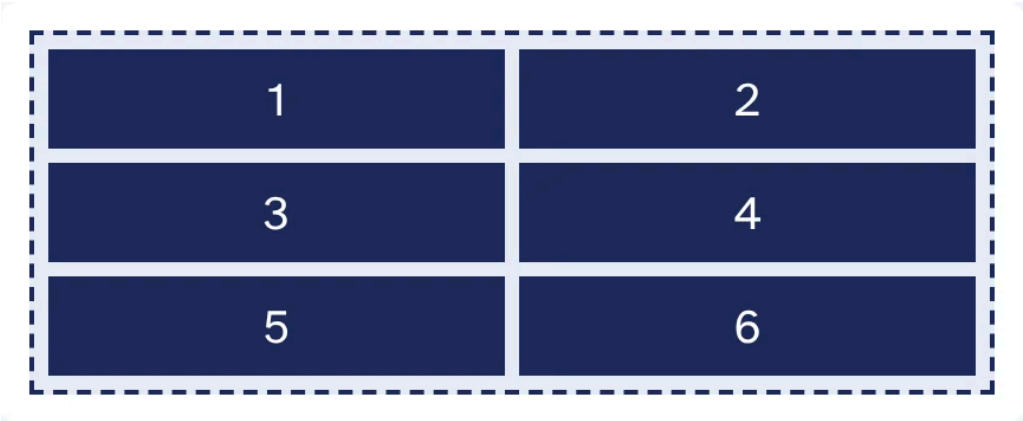


Figure 24. – `repeat(2, 1fr)`

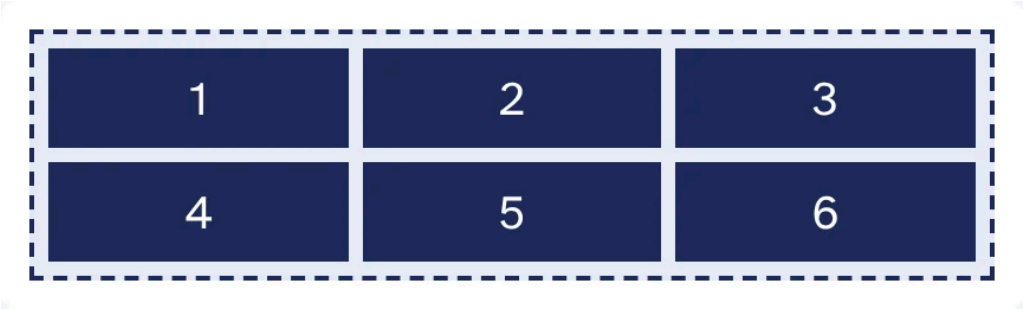


Figure 25. – `repeat(3, 1fr)`

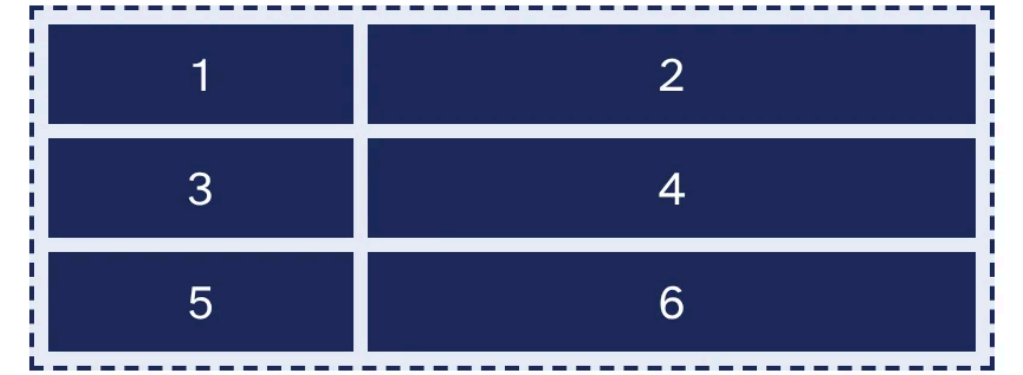


Figure 26. – `1fr 2fr`

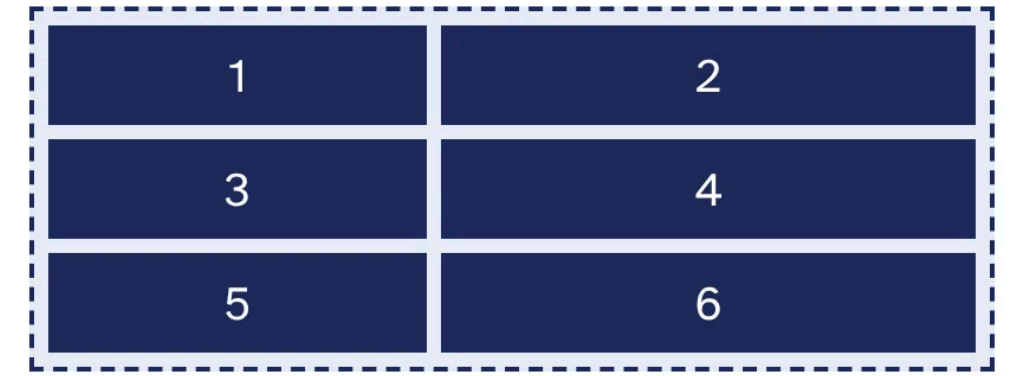


Figure 27. – `10rem 1fr`

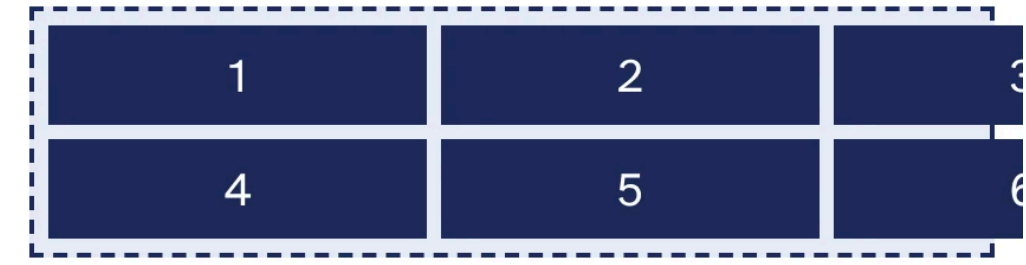


Figure 28. – `repeat(3, 10rem)`

Grid — attention et vérification

> ⚠ Attention
>
> Trois colonnes fixes sur un téléphone : les cartes deviennent minuscules. Une seule colonne au départ, plusieurs sur grand écran : c'est l'apport de cet après-midi.

- **Vérifiez** : Flexbox ou Grid pour une galerie de 9 photos ?

Lire une page de documentation — `justify-content` (20 min)

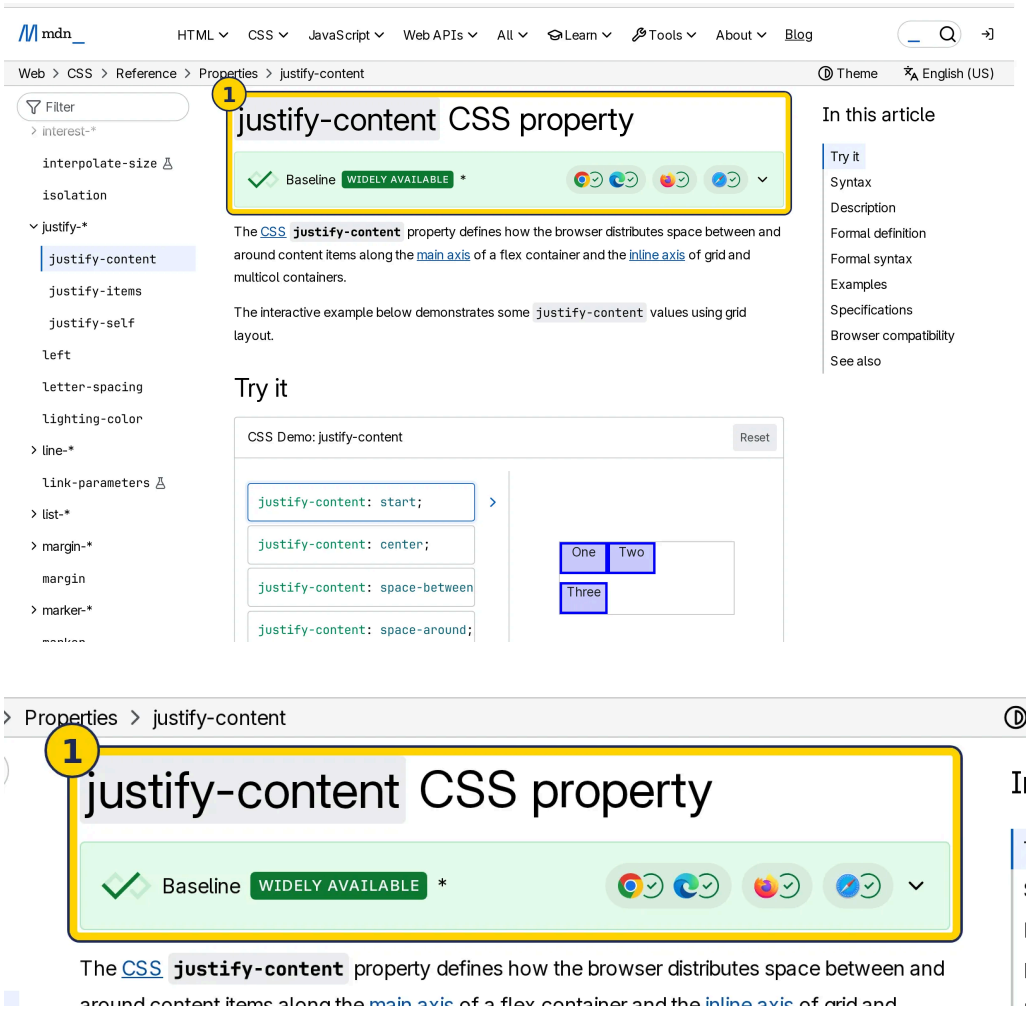
- Une page de référence MDN, en anglais : toujours le même plan.
- Syntaxe, valeurs, exemple, compatibilité : où les trouver.
- Puis l'essayer vous-même, dans VS Code.
- Quatre questions pour finir.

Niveau : guidé.

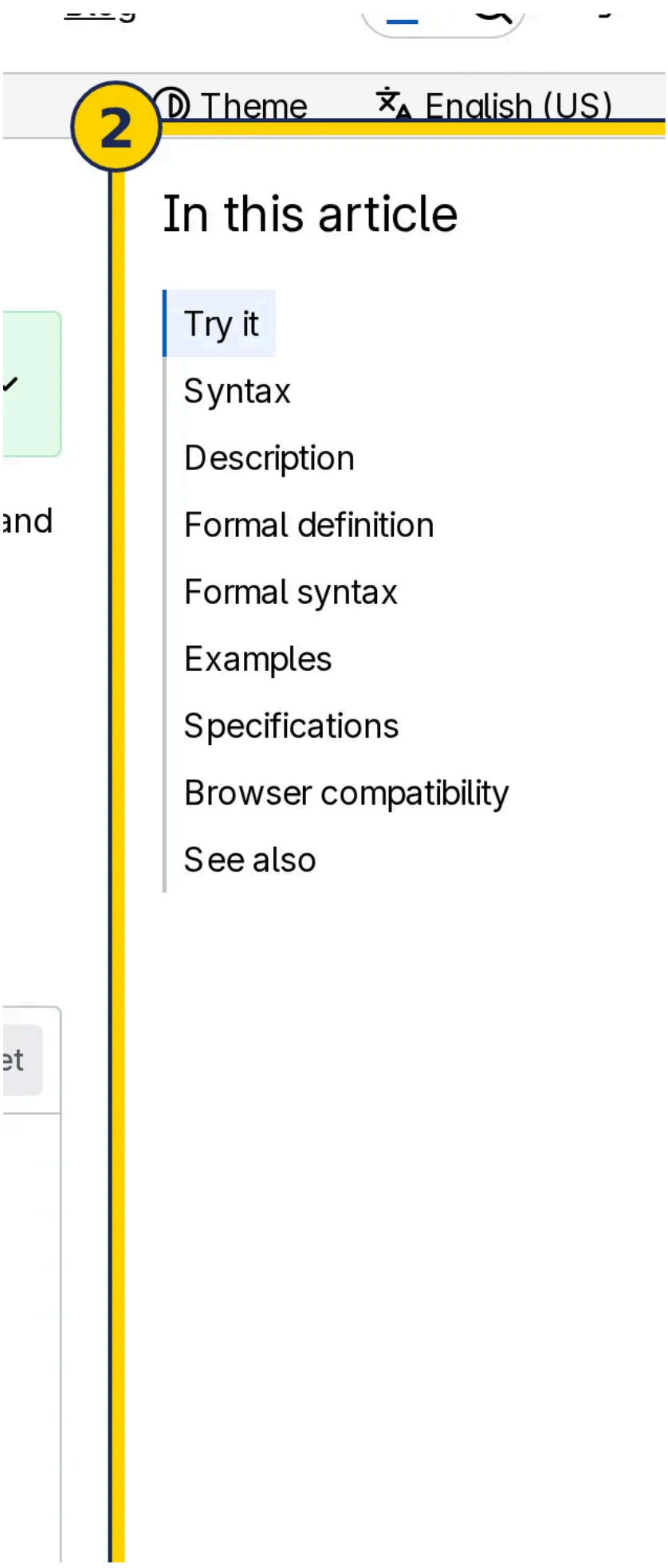
Une documentation de référence se lit comme un plan de ville : on ne la lit pas en entier, on sait où chercher. Toutes les pages de propriétés CSS de MDN suivent le même plan ; apprenez-le sur une propriété que vous connaissez depuis ce matin (MDN Web Docs, s. d.-c).

Pas à pas, en captures

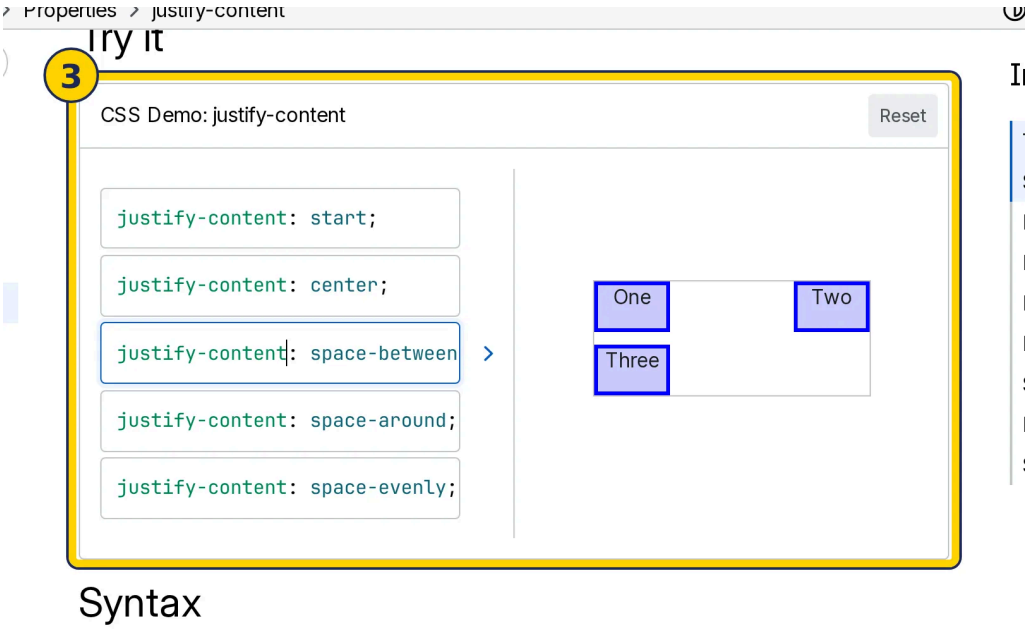
1. Ouvrez la page de `justify-content` sur MDN, en anglais : `developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/justify-content`. **Vous devez voir** : le titre « justify-content CSS property », puis un encadré vert *Baseline — Widely available* : la propriété marche dans tous les navigateurs récents.



2. Repérez le plan de la page, à droite : *In this article*. **Vous devez voir** : les rubriques dans l'ordre : *Try it*, *Syntax*, *Description*, *Formal definition*, *Formal syntax*, *Examples*, *Specifications*, *Browser compatibility*, *See also*. Un clic sur une rubrique y mène directement.


2
Theme
English (US)
In this article
Try it
Syntax
Description
Formal definition
Formal syntax
Examples
Specifications
Browser compatibility
See also

3. Rubrique *Try it* : cliquez sur la ligne `justify-content: space-between;`. **Vous devez voir** : à droite, les éléments se répartissent sur la largeur. Cet exemple interactif utilise une grille : `justify-content` marche aussi avec Grid.

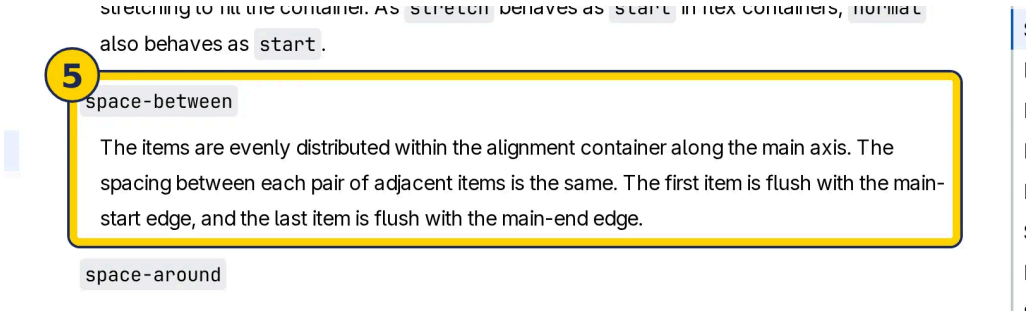


Syntax

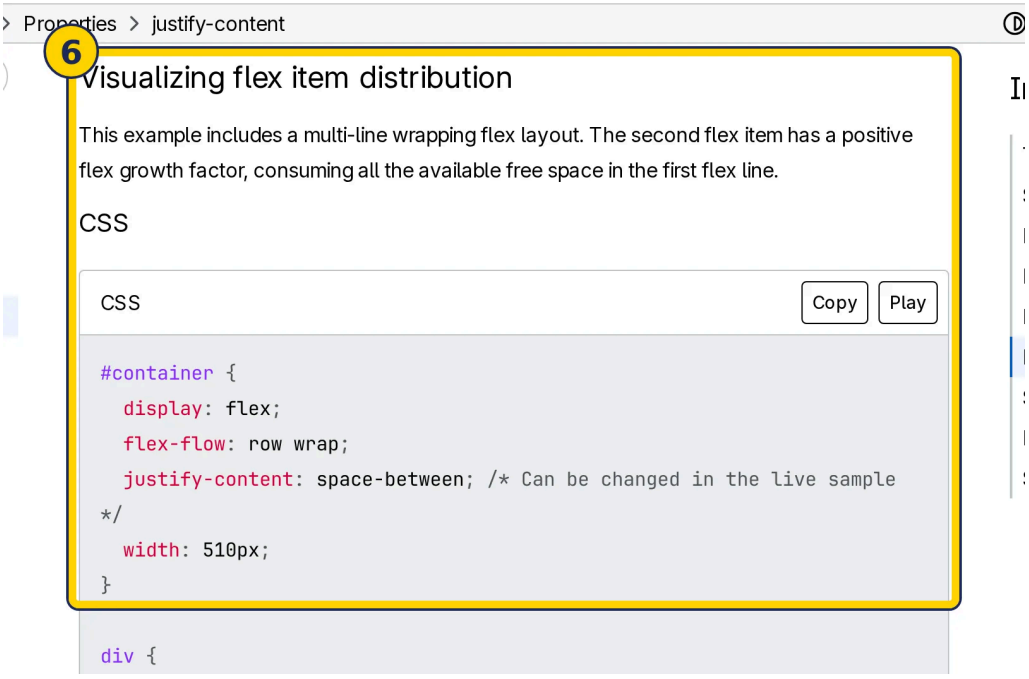
4. Rubrique *Syntax* : la liste des façons d'écrire la propriété. **Vous devez voir** : un bloc de code, une ligne par valeur (`justify-content: center;`, `justify-content: space-between;`…), rangées par familles, avec un commentaire au-dessus de chaque famille.



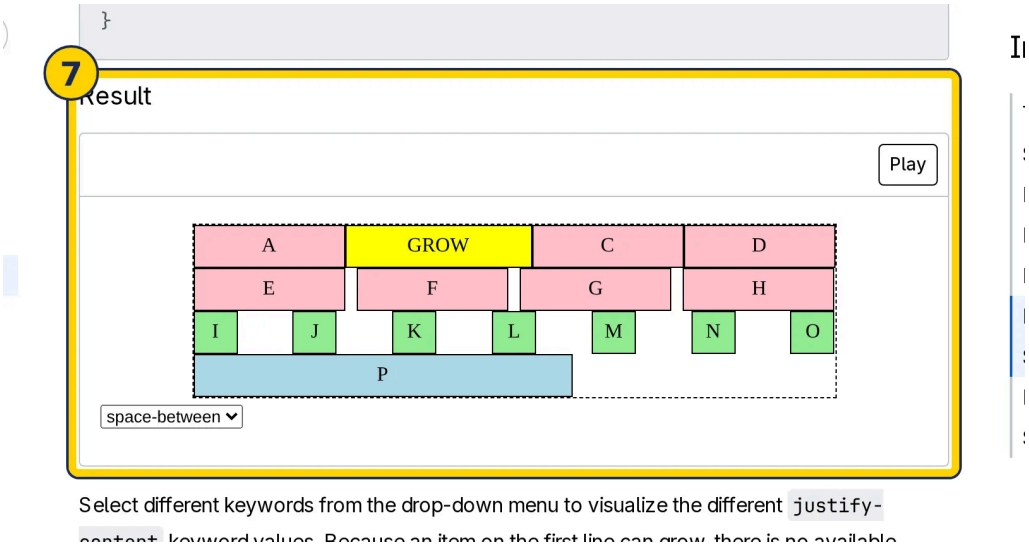
5. Sous-rubrique *Values* : cherchez `space-between`. **Vous devez voir** : le nom de la valeur, puis sa définition en anglais : les éléments sont répartis régulièrement ; le premier touche le bord de départ, le dernier le bord d'arrivée.



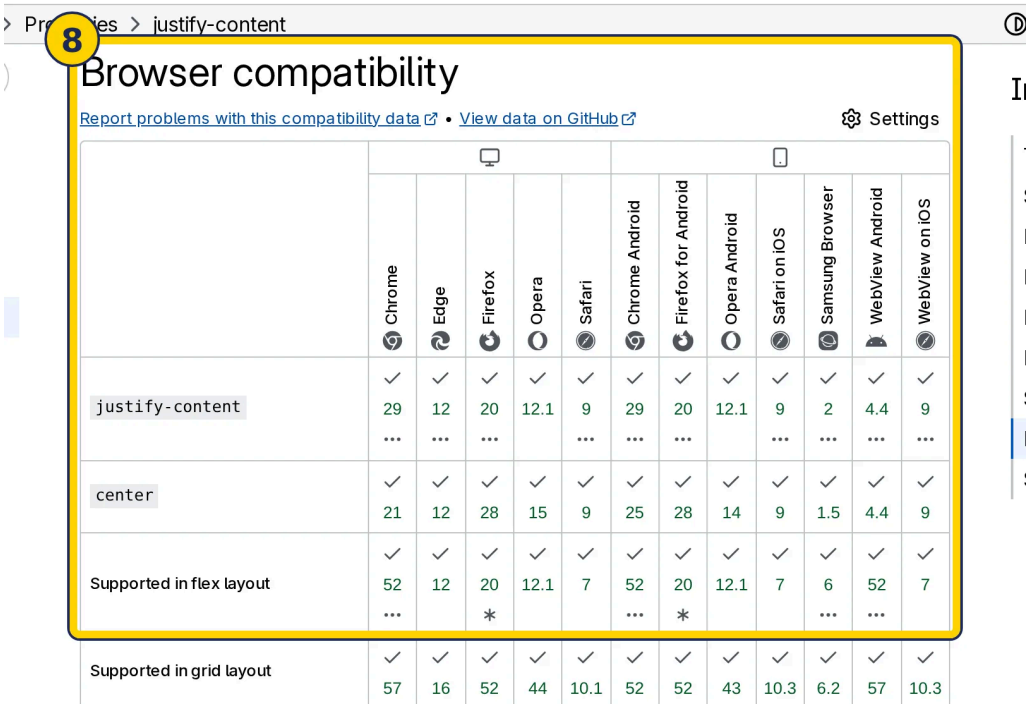
6. Rubrique *Examples*, sous-titre *Visualizing flex item distribution* : un exemple complet, avec son code. **Vous devez voir** : une phrase qui présente l'exemple, puis le code CSS, avec deux boutons : *Copy* (copier le code) et *Play* (l'ouvrir dans l'éditeur en ligne de MDN).



7. Juste en dessous, *Result* : le résultat de cet exemple. **Vous devez voir** : des cases colorées, réparties ligne par ligne, et une liste déroulante pour changer la valeur de `justify-content`.



8. Rubrique *Browser compatibility*, en bas de la page. **Vous devez voir** : un tableau : une colonne par navigateur, une ligne par usage. Une coche et un numéro : la version à partir de laquelle ce navigateur reconnaît la propriété. La ligne *Supported in flex layout* concerne Flexbox.



9. Essayez vous-même : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05`, créez le fichier avec `touch essai-justify.html` ; dans VS Code, écrivez-y le code ci-dessous, et ouvrez-le avec Live Preview. Remplacez `space-between` par chaque valeur de la liste *Values* (`center`, `space-around`, `space-evenly`…), en enregistrant à chaque fois. **Vous devez voir** : à chaque enregistrement, les trois liens changent de place dans l'aperçu, comme dans l'exemple de MDN.

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Essai de justify-content</title>
  <style>
    nav { display: flex; justify-content: space-
between; border: 2px dashed; }
  </style>
</head>
<body>
  <nav><a href="#">Accueil</a> <a href="#">Projets</a>
<a href="#">Contact</a></nav>
```

```
</body>
</html>
```

Les publicités affichées par MDN ont été retirées des captures : vous les verrez peut-être sur la page.

Quatre questions, en anglais dans le texte
Répondez en français, en citant la rubrique où vous avez trouvé la réponse.

1. Sur quel axe `justify-content` répartit-il les éléments d'un conteneur flex ?
2. Que fait `space-between` ? Et `space-evenly` ?
3. Quelle valeur place tous les éléments au centre ?
4. À partir de quelle version de Chrome `justify-content` marche-t-il avec Flexbox ?

Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add essai-justify.html
git commit -m "Essaie les valeurs de justify-content"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

À retenir — ce matin

> À retenir
>
> - **Flexbox** : une dimension ; `display: flex` sur le parent.
> - **Grid** : deux dimensions ; `grid-template-columns` et `fr`.

Le détail, notion par notion :

- **Flexbox** : `justify-content` sur l'axe principal, `align-items` sur l'axe secondaire, `gap` entre les éléments ; toujours sur le parent.
- **Grid** : des colonnes avec `grid-template-columns`, `1fr` partage la place ; Flexbox pour une rangée, Grid pour une grille.

Si le temps le permet — Les derniers niveaux de Froggy

1. Terminez les niveaux restants de Flexbox Froggy.

> Défi
>
> Expliquez à un voisin la différence entre `align-items` et `align-content`, avec un niveau de Froggy comme exemple.

> Défi
>
> Le terrain d'essai Flexbox et Grid : changez chaque réglage, prédisez le résultat **avant** de regarder, puis recopiez dans votre page le code affiché qui vous sert (15 min).

Débrief

- Quand choisir Flexbox, quand choisir Grid ?
- Où trouver les valeurs d'une propriété sur une page de MDN ?

Après-midi — TP — Grid Garden, puis « Layout Lab »

Grid Garden (45 min)

1. Ouvrez Grid Garden, version du cours : le jeu s'affiche en français.
2. Arrosez les carottes en plaçant le terrain en CSS Grid.
3. **Réussi si** : le niveau 12 au moins (28 pour les plus rapides).
4. Capturez le dernier niveau réussi.

Niveau : guidé.

Grid Garden existe en français et compte 28 niveaux (Thomas Park, s. d.-b). Cette copie est hébergée avec le cours, sans mesure d'audience ; votre progression reste dans votre navigateur. Code : MIT, © Thomas Park. Images : Creative Commons BY 3.0, © Thomas Park.

Apport — Mobile first (15 min)

- **Pourquoi** : la majorité des visiteurs arrivent sur un téléphone.
- **Mobile first** : les styles de base pour le téléphone, puis on enrichit.

Pourquoi. Commencer par le petit écran oblige à décider ce qui compte ; le grand écran ajoute de la place, pas de nouvelles idées.

Définition. En mobile first, les styles de base visent le téléphone (une seule colonne) ; une media query en `min-width` ajoute les styles des grands écrans (MDN Web Docs, s. d.-e). La balise `meta name="viewport"` de lundi est indispensable pour que le téléphone utilise sa vraie largeur (MDN Web Docs, s. d.-f).

Exemple.

```
.projets { display: grid; grid-template-columns: 1fr; }

@media (min-width: 48rem) {
  .projets { grid-template-columns: repeat(3, 1fr); }
}
```

Résultat : une page qui utilise ce code (un menu en Flexbox, trois projets en Grid, la media query `min-width: 48rem`), à trois largeurs.



Figure 29. – 375 px, un téléphone : une colonne



Figure 30. – 768 px, une tablette : la media query s'applique, trois colonnes



Figure 31. – 1280 px, un ordinateur : trois colonnes, contenu centré

Pas à pas.

1. Écrivez la règle de base : une colonne.
2. F12, mode responsive : réglez la largeur à 375 px ; une colonne.
3. Ajoutez la media query ; élargissez à plus de 768 px (48 rem) : trois colonnes.

Mobile first — attention et vérification

> ⚠ Attention
>
> Une media query en `max-width` (héritée d'un tutoriel) inverse la logique : le téléphone reçoit des corrections au lieu des styles de base. Ici, toujours `min-width`.

- **Vérifiez** : à 375 px, combien de colonnes dans l'exemple ?

« Layout Lab » (1 h 30)

1. Mettez en page votre page personnelle, en *mobile first*.
2. En-tête et menu : Flexbox ; projets ou compétences : Grid.
3. Une media query en `min-width` pour les grands écrans.
4. Vérifiez à 375 px avec le mode responsive des outils (F12).
5. **Réussi si** : `node --test layout.test.js` passe.

Niveau : semi-guidé.

Enregistrer et envoyer (2 min)

Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add page-perso/style.css
git commit -m "Met en page la page personnelle en
mobile first"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

Publier sur GitHub Pages (20 min)

1. Un dépôt **public** à part, `page-perso` : `dwwm-tp` reste privé.
2. `git clone`, puis copie de `index.html` et `style.css` ; `git push`.
3. *Settings* → *Pages* → *Deploy from a branch* : `main`, `/ (root)`.
4. Après deux minutes : `pseudo.github.io/page-perso/`.
5. **Réussi si** : la page s'affiche, avec le cadenas (HTTPS).

Niveau : guidé.

> ⚠ Attention
>
> Un dépôt **public** est visible par tout le monde, et ses fichiers peuvent être copiés et indexés par les moteurs de recherche. Aucune donnée personnelle dedans : prénom ou pseudonyme, aucune adresse e-mail, aucun numéro de téléphone, aucune adresse postale. `dwwm-tp` reste privé : vous n'y changez rien.

GitHub Pages publie un site à partir d'un dépôt ; avec un compte gratuit, le dépôt doit être public (GitHub, s. d.-a). C'est pourquoi la page part dans un dépôt à part, `page-perso`, qui ne contient que les deux fichiers de la page.

1. Sur GitHub, *New repository* : nom `page-perso`, **Public**, sans README.
2. Dans Git Bash, en remplaçant `pseudo` par le vôtre :

   ```
   cd ~/dwwm
   git clone git@github.com:pseudo/page-perso.git
   cp dwwm-tp/semaine-05/page-perso/index.html dwwm-tp/
   semaine-05/page-perso/style.css page-perso/
   cd page-perso
   git add index.html style.css
   git commit -m "Publie la page personnelle"
   git push
   ```

   Vous devez voir : à la fin, `* [new branch]  main -> main` ; sur GitHub, les deux fichiers dans `page-perso`.
3. Puis *Settings* → *Pages*, comme sur la diapositive.

Vous continuez à travailler dans `dwwm-tp/semaine-05/page-perso` (les tests y sont). Pour mettre la page publiée à jour, refaites la copie (`cp`), puis `git add`, `git commit` et `git push` depuis `~/dwwm/page-perso`. Les sites de projet ont une adresse de la forme `https://<pseudo>.github.io/<dépôt>` (GitHub, s. d.-c) et peuvent imposer HTTPS (GitHub, s. d.-b). La page n'est pas finie ? Publiez quand même : vous la corrigerez demain, après les audits, et chaque `push` met la page publiée à jour. Avant de pousser, relisez

la page : prénom ou pseudonyme, aucune adresse e-mail en clair, formulaire annoncé comme non fonctionnel.

À retenir — cet après-midi

> À retenir
>
> - **Mobile first** : une colonne, puis `@media (min-width)`.
> - **Mise en page** : vérifiez à 375 px, sans barre de défilement horizontale.
> - **GitHub Pages** : un dépôt public à part, *Deploy from a branch*, `main`.

Critères de réussite

- □ Flexbox Froggy : 24 niveaux ; Grid Garden : 12 au moins
- □ `node --test layout.test.js` : tout au vert
- □ Aucune barre de défilement horizontale à 375 px
- □ La page est publiée sur GitHub Pages, en HTTPS

Si le temps le permet — Publier le défi au portfolio

- Même méthode, un dépôt public : le composant de mercredi.
- Puis un `README.md` en anglais : lien publié, trois écarts corrigés.
- Pas fini ? Vendredi après-midi, ou au soutien de mardi.

Si le temps le permet — Le formulaire sur grand écran

> Défi
>
> - Sur grand écran, placez chaque `label` à gauche de son champ.
> - Une grille à deux colonnes, dans la media query.

Pour aller plus loin

- Découvrez `grid-template-areas` : nommez les zones de la page.
- Reconstruisez la mise en page avec ces noms.

Sources

GitHub. s. d.-a. « Creating a GitHub Pages site ». Consulté le 26 septembre 2026. https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site.

GitHub. s. d.-b. « Securing your GitHub Pages site with HTTPS ». Consulté le 26 septembre 2026. https://docs.github.com/en/pages/getting-started-with-github-pages/securing-your-github-pages-site-with-https.

GitHub. s. d.-c. « What is GitHub Pages? » Consulté le 26 septembre 2026. https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages.

MDN Web Docs. s. d.-a. « Boîtes flexibles - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Flexbox.

MDN Web Docs. s. d.-b. « Grilles - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Grids.

MDN Web Docs. s. d.-c. « justify-content CSS property ». Consulté le 23 octobre 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/justify-content.

MDN Web Docs. s. d.-d. « Les concepts de base des grilles - CSS ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Guides/Grid_layout/Basic_concepts.

MDN Web Docs. s. d.-e. « Media queries - CSS ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Guides/Media_queries/Using.

MDN Web Docs. s. d.-f. « Responsive design - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Responsive_Design.

Thomas Park. s. d.-a. « Flexbox Froggy - A game for learning CSS flexbox ». Consulté le 26 septembre 2026. https://flexboxfroggy.com/#fr.

Thomas Park. s. d.-b. « Grid Garden - A game for learning CSS grid layout ». Consulté le 26 septembre 2026. https://cssgridgarden.com/#fr.